Pontificia Universidad Católica de Chile
Departamento de Ciencia de la Computación
IIC2613 - Inteligencia Artificial

Reflexión

15 de junio de 2026

1º semestre 2026

Rafael Lorca - 23641789

DCContainer Marshalling

Parte 1

2.4 Representación de acciones y cambio

- ¿Cuál es el efecto positivo de la acción move(C1,C2,T)?
  Los efectos positivos son los hechos que se agregan al estado del mundo luego de hacer una acción, en este caso move(C1,C2,T) significa que el containter C1 se mueve sobre el container C2 en el tiempo T, luego, el efecto positivo sería on(C1,C2,T+1), ya que en el tiempo siguiente el container C1, está sobre el container C2.
- ¿Cuál es el efecto negativo de la acción move(C1,C2,T)?
  Los efectos negativos son los hechos que se eliminan al estado del mundo luego de hacer una acción, en este caso move(C1,C2,T) tiene como efecto negativo (suponiendo que C! estaba sobre otro container C3 en el tiempo T) on(C1,C3,T+1).
- ¿Cuál es la precondición de esta acción?
  Las precondiciones son hechos que tienen que ser verdaderos para que la acción pueda ejecutarse. La precondición de move(C1,C2,T) es que encima del container C2 no haya otro container, lo mismo para el container C1, no tiene que tener otro container encima.

Decisiones de modelamiento

Para modelar la dinámica del predicado on se optó por el patrón estándar de inercia en ASP: en vez de derivar explícitamente un efecto negativo (lo cual no es posible en ASP, ya que not solo puede aparecer en el cuerpo de una regla), se condiciona la persistencia de on(C1,C2,T+1) a que no haya ocurrido un $move(C1, _, T)$ en ese paso de tiempo.

Para representar qué containers están disponibles como destino se definieron los predicados auxiliares ocupado y libre, donde un container está libre si está en el patio y no tiene ningún otro container directamente encima. Esto permite expresar las precondiciones de move de forma limpia. Adicionalmente, se definió valido(D,T) para unificar los destinos legales: una base de pila desocupada o un container libre. El predicado ordenado(T) se define como la negación por falla de desordenado(T), donde desordenado(T) se deriva cuando existe un container de mayor prioridad directamente sobre uno de menor prioridad. Esta decisión es natural en ASP: es más directo describir cuándo algo está mal que cuándo está bien.

Para el Desafío 4, se introdujo el predicado $en_pila(C, B, T)$ de forma recursiva para rastrear qué containers pertenecen a la pila que nace desde la base B, lo que permitió definir tope(B,X,T) como el elemento libre en esa pila, necesario para modelar correctamente el efecto de drop. Otros predicados que se introdujeron en este desafío es el de apilado(B,T), que dice si una pila tiene algo aparte de la base.

Para las restricciones del Desafío 4, se cumple FIFO con una restricción que elimina del modelo aquellos que dropean un container C2 antes que otro container C1, si es que el tiempo en que ingresó C1 al área adyacente al patio es menor que el tiempo en que entró C2 (entró antes). Para el no paralelismo, sacamos de nuestro modelo aquellos que tengan un movimiento de containers y un drop en el mismo paso T.

Pruebas realizadas

Se utilizó IA para hacer algunos tests extras, el promt fue: Creame unos casos límite que podrían generar problemas siguiendo el formato entregado y haciendo que funcionen con el visualizador (le di el archivo visualizador).

En el desafío 1:

- test1: Satisfacible, la visualización corre bien.
- test2: Satisfacible, la visualización corre bien.
- test3: Satisfacible, la visualización corre bien.
- test4: Satisfacible, pero no tenía nada por lo que la visualización no corrió.

En el desafío 2:

- test1: Satisfacible, la visualización corre bien.
- test2: Satisfacible, la visualización corre bien, aunque cambié un poco el test para ver si se movía el container de arriba bien. (Añadí la linea move(1,2,1), por lo que decía la descripción del test, parecía que faltaba algo así).
- test3: UNSAT (que está bien).
- test4: Parecido al anterior, es un test donde se intenta mover un container que no está libre. UNSAT.

En el desafío 3:

- test1: Satisfacible, la visualización corre bien (trivial).
- test2: Satisfacible, la visualización corre bien, eso sí, lo que hace es eliminar un container para ordenar los que están encima.
- test3: Satisfacible, la visualización corre bien. Lo mismo que el anterior, elimina containers y también hace un movimiento extra.
- test4: Satisfacible aunque ocurre lo mismo que antes. Quería ver que pasaba en un caso donde había una columna con altura en el límite y se comporta bien (baja los containers a otras pilas).
- test5: UNSAT con bound 1, pero sat con bound 2, partían en el límite de altura pero como nuestro solver borra containers, con un tiempo más resulta satisfacible luego de eliminar un container.

En el desafío 4:

- test1: Satisfacible, corre perfecto.
- test2: Satisfacible, era para ver FIFO, se resoeta que entra 3 antes que 2.
- test3: Satisfacible, se ve el no paralelismo super bien.

Dificultades encontradas

Una de las principales dificultades fue el manejo de variables no ligadas (unsafe variables) en ASP. En particular, al definir libre inicialmente se intentó usar not on(C2,C1,T) directamente, pero C2 no estaba ligada a ningún dominio, lo que generaba errores. La solución fue definir primero ocupado(C1,T) ligando C2 mediante container(C2), y luego definir libre en términos de ocupado. Esto llevó a entender que en ASP es natural y correcto definir predicados auxiliares intermedios, no es un workaround sino parte del paradigma.

Otra dificultad fue entender que en ASP no existe el efecto negativo explícito: no se puede escribir not on(C1,C2,T+1) como cabeza de una regla. Esto requirió reformular completamente la lógica de persistencia, pasando de pensar en "qué deja de ser verdad.ª pensar en "bajo qué condiciones algo sigue siendo verdad".

Reflexión

Considero que ASP resulta particularmente adecuado para problemas de planificación como el Container Marshalling porque nos permite describir bastante bien el problema, se especifican restricciones lógicas y que pueden ser modeladas con ASP, de cierta manera bosquejamos una via de solución y el solver se encarga de encontrarla. Esto contrasta con un enfoque imperativo donde habría que implementar explícitamente un algoritmo de búsqueda. En este problema, la combinación de choice rules para generar posibles acciones, restricciones

de integridad para eliminar configuraciones inválidas, y el las reglas para representar los pasos o momentos de tiempo, capturan de forma elegante la complejidad del dominio. La principal limitación que vi es que el horizonte temporal debe definirse de antemano mediante bound, lo que puede llevar a instancias sin solución si el bound es muy pequeño, o a tiempos de cómputo elevados si es muy grande.